HANGZHOU DIANZI UNIVERSITY

Information Security Laboratory

LABORATORY WORK No. 1

Secure REST API Development with CI/CD and Real CVE Analysis

Variant: CVE-2017-5638 · Apache Struts 2 (S2-045)

Student **Pan Hongzhou**

HDU number **23320333**

Variant **CVE-2017-5638**

Date **19 October 2026**

All three stages are complete and verified locally.
The public CI run is green (3/3 jobs, 42 s).

1. Purpose

The purpose of this laboratory work is to go through the full security lifecycle of a backend service: design and build a small REST API, protect it against the OWASP Top 10 classes required by the assignment, automate security checks in a CI/CD pipeline, and then study a real vulnerability in a controlled environment — reproducing it, explaining its root cause, applying a fix and proving that the fix works.

The specific objectives were:

- **Functional API —** implement a functional API with authentication, protected data retrieval and one self-designed method;
- **Security —** prevent SQL injection, cross-site scripting and broken authentication;
- **CI/CD —** run SAST and SCA automatically on every push and publish their reports;
- **CVE analysis —** reproduce a real CVE, analyse its cause and verify remediation.

This report documents all three stages: the API and its security controls, the CI/CD pipeline, and the reproduction and remediation of CVE-2017-5638 against a Vulhub environment. The CVE experiment was executed on 19 October 2026 on Docker Desktop 29.8.2 with WSL 2.6.1; the observed commands and output are quoted from that run.

2. Scope and Status

Component	Status	Evidence
Secure REST API (4 endpoints)	Complete	Flask application, tested over HTTP
SQLi / XSS / authentication controls	Complete	Implementation + 59 automated tests
CI/CD with SAST and SCA	Complete	.github/workflows/ci.yml
Remote pipeline run and screenshots	Complete	Run #1: 3/3 jobs green in 42 s
CVE-2017-5638 reproduction	Complete	Vulhub Struts 2.3.30; OGNL and id execution
Fix and regression verification	Complete	2.3.32 build rejects the same exploit

Being explicit about this boundary matters: no screenshot or command output in this report is invented. Everything marked complete was executed or captured on 19 October 2026; the run page, artifacts, console screenshots and raw reports are linked from the repository's README and docs/evidence/ directory.

3. Architecture and Technology Stack

The service is a stateless Flask application. Clients authenticate once and then send a JWT bearer token with every request. The API validates the token, extracts the user identity, and reaches SQLite exclusively through parameterized statements. The CI pipeline runs independently on GitHub Actions and publishes security reports as build artifacts.

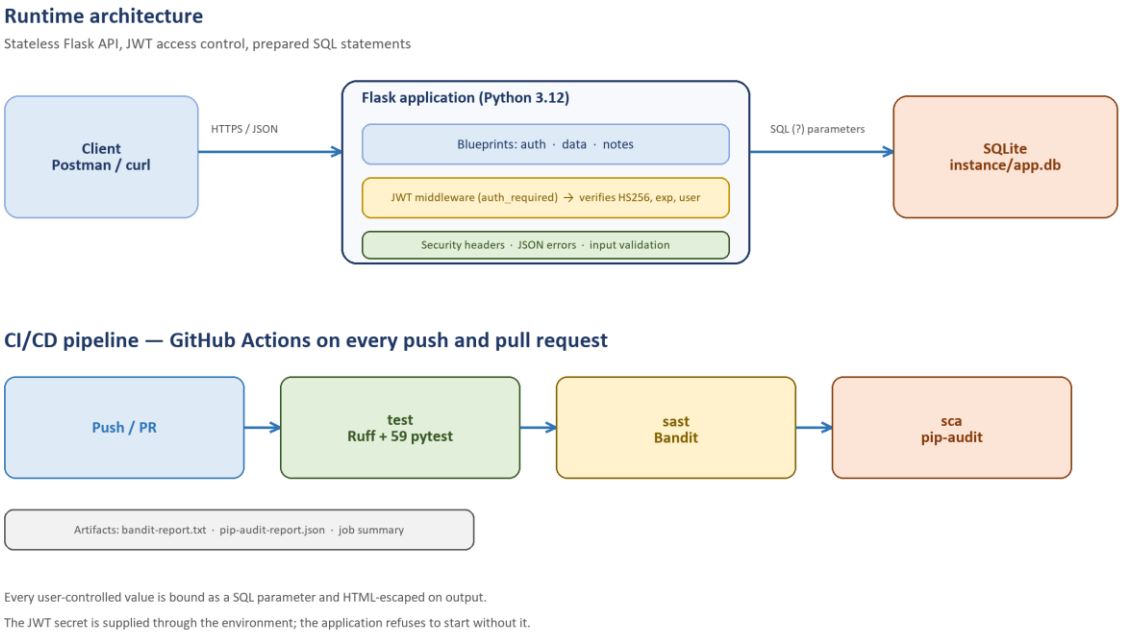


Figure 1. Runtime architecture and CI/CD pipeline.

Layer	Technology	Reason
Language / framework	Python 3.12, Flask 3.1	Small, auditable code base
Database	SQLite (sqlite3)	No setup; real prepared statements
Authentication	PyJWT 2.15 (HS256), bcrypt 5.0	Required by the assignment
SAST	Bandit 1.9	Python-specific security linting
SCA	pip-audit 2.10	Checks pinned dependencies against advisories
CI/CD	GitHub Actions	Runs on every push and pull request

Table 1. Technology choices.

4. API Implementation

4.1 Endpoints

Method	Path	Auth	Purpose
POST	/auth/login	No	Authenticate and issue a JWT
GET	/api/data	Yes	Return the user directory; optional ?q= filter
POST	/api/notes	Yes	Create a note owned by the caller
GET	/api/notes	Yes	List only the caller's notes

POST /api/notes is the self-designed third method. It was chosen because it stores and returns free-form user text, which makes the XSS protection visible and testable; it also demonstrates per-user authorization because every note is bound to the identity from the token.

4.2 Authentication flow

1. **Login —** the client sends username and password to POST /auth/login;
2. **Verification —** the server looks the user up with a parameterized query and verifies the password with bcrypt;
3. **Token issue —** on success it signs a JWT (HS256) containing sub, username, role, iat, exp and jti;
4. **Use —** the client sends Authorization: Bearer <token> on protected requests;
5. **Middleware —** auth_required verifies the signature, algorithm, expiry and required claims, then reloads the user and attaches it to the request context.

4.3 Data model

Table	Columns	Notes
users	id, username, password_hash, role, bio	username UNIQUE; hash is bcrypt
notes	id, user_id, title, content, created_at	foreign key to users; indexed by user_id

4.4 Example requests

```
curl -s -X POST http://127.0.0.1:5000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"ChangeMe#2024"}'

TOKEN="<access_token>"
curl -s http://127.0.0.1:5000/api/data -H "Authorization: Bearer $TOKEN"

curl -s -X POST http://127.0.0.1:5000/api/notes \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"title":"Lab notes","content":"Prepared statements everywhere."}'
```

5. Security Measures

5.1 SQL injection protection

Every SQL statement is a static string with ? placeholders, and user input is passed as a bound parameter. No query is assembled by concatenation or f-strings, which removes the parser-level confusion that SQL injection relies on. The optional search filter is parameterized as well; even the % wildcards belong to the bound value.

```
db.execute(
    "SELECT id, username, password_hash, role, bio "
    "FROM users WHERE username = ?",
    (username,),
)
```

Tests send ' OR '1'='1, alice'--, '; DROP TABLE users; -- and UNION-based payloads to both the login and the search parameter. Every attempt fails (401 or zero rows), and a legitimate login still succeeds afterwards, proving the schema was not altered.

5.2 XSS protection

All user-controlled text is HTML-escaped when it is serialized, so <script>alert(1)</script> leaves the API as <script>alert(1)</script>. The database keeps the original text; only the representation is encoded. This follows the output-encoding pattern required by the assignment and works together with the application/json content type, a restrictive Content-Security-Policy and nosniff.

```
def escape_text(value: object) -> str:
    if not isinstance(value, str):
        return ""
    return html.escape(value, quote=True)
```

Automated tests create notes containing <script> and <img onerror> payloads and assert that the raw tags never appear in the create or list responses while the escaped form does.

5.3 Authentication and password storage

- **Passwords —** bcrypt with a random salt and cost 12; passwords longer than 72 bytes are rejected; plaintext is never stored or logged.
- **JWT —** HS256 with a secret of at least 32 characters, one-hour lifetime, and pinned verification (alg: none and algorithm confusion are rejected).
- **Middleware —** the token is verified on every protected request, the subject is resolved to an existing account, and any failure returns 401.
- **Enumeration —** unknown user and wrong password produce the same response; a dummy bcrypt check equalizes timing.

```
jwt.decode(
    token,
    current_app.config["JWT_SECRET"],
    algorithms=["HS256"],
    options={"require": ["exp", "iat", "sub"]},
)
```

5.4 Additional controls

Control	Implementation
Secret management	JWT_SECRET and SEED_PASSWORD come from the environment; the app refuses to start without a 32+ character secret
Request size	MAX_CONTENT_LENGTH = 16 KiB, returns JSON 413
Input validation	types and lengths checked before use (username 64, query 64, title 120, content 5000)
Error handling	JSON 400/401/404/405/413/500 without stack traces or SQL details
Tenant isolation	notes are filtered by the user_id from the verified token
Headers	CSP default-src 'none', nosniff, DENY framing, no-referrer, no-store

5.5 Verification

The test suite contains 59 tests across four files. They cover the happy path and the attacks: SQL injection in login and search, stored XSS, missing/expired/tampered/unsigned tokens, deleted users, malformed headers, validation boundaries and security headers.

Test file	Focus	Tests
tests/test_auth.py	login, validation, JWT attacks, deleted user	27
tests/test_data.py	access control, data exposure, SQLi-safe search	8
tests/test_notes.py	XSS escaping, isolation, SQLi in content	16
tests/test_security.py	headers, bcrypt storage, JSON errors, size limit	8

```
$ python -m pytest
59 passed in 1.79s

$ python -m bandit -r app -f txt
No issues identified.

$ python -m pip_audit --desc
No known vulnerabilities found
```

A live smoke test additionally exercised the running server over HTTP: 13 of 13 checks passed, including the 401 without a token, successful login, rejected SQLi login, escaped XSS payload and rejected unauthenticated note listing.

6. CI/CD Pipeline

The workflow .github/workflows/ci.yml runs on every push to any branch and on every pull request. It has three jobs, all on Python 3.12: test, sast and sca. The two security jobs depend on the test job, so a broken build is not scanned and a failing scanner blocks the pipeline.

Job	Steps	Output
test	install pinned dependencies, ruff check ., pytest	test results
sast	bandit -r app -f txt	bandit-report.txt artifact + job summary
sca	pip-audit -r requirements.txt (console and JSON)	pip-audit-report.json artifact + job summary

Both reports are appended to the GitHub Actions run summary, which is the most convenient page to screenshot. The artifacts are retained by GitHub and can be downloaded from the run page.

Public CI run

The first Actions run on commit e2fd37b completed with 3/3 jobs green in 42 s: test 17 s, SAST (Bandit) 16 s and SCA (pip-audit) 19 s. The run page and the two artifacts are reproduced in Appendix D.

7. CVE Analysis: CVE-2017-5638 (Struts 2 S2-045)

Execution status: completed on 19 October 2026

The Vulhub directory struts2/s2-045 (Struts 2.3.30) was launched with Docker Compose. The vulnerability was reproduced both with the official OGNL proof and with an id command, then a rebuilt 2.3.32 image was used to show that the same requests no longer execute.

7.1 Vulnerability summary

Field	Value
CVE	CVE-2017-5638 (Apache advisory S2-045)
Product	Apache Struts 2 — Jakarta Multipart parser
Affected	2.3.5 – 2.3.31 and 2.5 – 2.5.10
Fixed	2.3.32 / 2.5.10.1
CVSS v3	10.0 — network, no privileges, no user interaction
Impact	Unauthenticated remote code execution

7.2 Reproduction and observed result

The official Vulhub proof sends a multipart POST whose Content-Type contains an OGNL expression that adds a response header with the value 233*233. Because the header is evaluated by the server, the response reveals the result of the calculation:

```
curl -i -s -X POST http://192.168.3.9:8080/ \
  -H "Content-Type: %{#context['com.opensymphony.xwork2.dispatcher.HttpServletResponse']
      .addHeader('vulhub',233*233)}.multipart/form-data" \
  --data-binary ""
```

The response was 200 OK with the header vulhub: 54289 (233^2), which proves arbitrary OGNL evaluation. Replacing the expression with a ProcessBuilder payload that runs id returned:

```
uid=0(root) gid=0(root) groups=0(root)
```

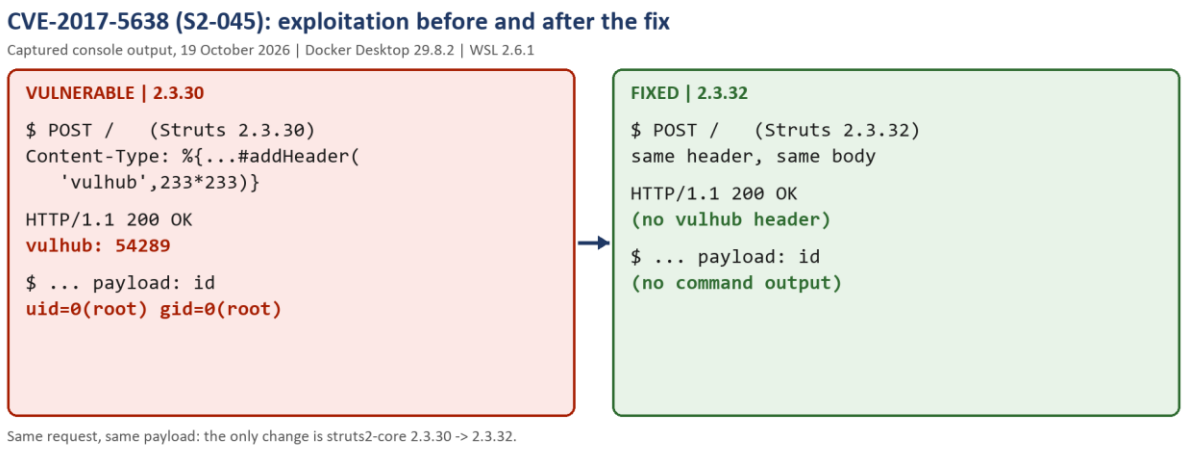


Figure 2. CVE-2017-5638 on the vulnerable and fixed builds (captured console output).

7.3 Root cause

A multipart request reaches JakartaMultiPartRequest.parseRequest(). When the Content-Type header is malformed, the parser builds an error message and localizes it through LocalizedTextUtil.findText(...), passing the raw header value as a message parameter. Struts 2 message rendering then calls TextParseUtil.translateVariables(), which finds %{...} sequences and evaluates their content as OGNL. The attacker therefore controls an expression that is evaluated on the server; clearing the context restrictions and using ProcessBuilder turns it into arbitrary command execution.

In one sentence: untrusted request metadata is fed into a message-localization path that performs OGNL evaluation, so the header becomes code. This is an expression-language injection (CWE-917), caused by improper input handling rather than a memory-safety bug or a configuration mistake.

7.4 Fix

The fix is a version upgrade, not a filter. cve/s2-045/Dockerfile.fixed starts from the vulnerable image and changes the dependency in /usr/src/pom.xml from 2.3.30 to 2.3.32, then removes the cached vulnerable jar and the old build output:

```
FROM vulhub/struts2:2.3.30
WORKDIR /usr/src
RUN sed -i 's|<version>2.3.30</version>|<version>2.3.32</version>|' pom.xml \
    && rm -rf /root/.m2/repository/org/apache/struts/struts2-core/2.3.30 \
    && rm -rf target
```

2.3.32 is the first 2.3.x release containing the S2-045 fix. The patched message-localization path no longer evaluates untrusted message parameters as OGNL, so a Content-Type containing %{...} is treated as plain text. The fixed build is started with docker compose -f docker-compose.fixed.yml build && docker compose -f docker-compose.fixed.yml up -d.

7.5 Verification

Check	2.3.30 (vulnerable)	2.3.32 (fixed)
vulhub response header	vulhub: 54289	absent
id command output	uid=0(root) ...	absent
Normal upload page	HTTP 200	HTTP 200
Loaded struts2-core	2.3.30	2.3.32

The same two requests were replayed against the fixed build on port 8081. Both returned the normal Struts2 Showcase file-upload page, with no vulhub header and no command output. Regression testing confirmed that the page still loads with HTTP 200, and the fixed container reports struts2-core-2.3.32.jar in its local Maven repository.

Environment note: on this machine 127.0.0.1:8080 was already occupied by a local Chromium/CEF debugging listener, so the vulnerable container was reached through the LAN address 192.168.3.9:8080 and the fixed build was published on 8081. The port is not part of the vulnerability; on a clean host localhost:8080 works as documented by Vulhub.

8. Compliance with the Assignment

Requirement	Where it is satisfied
Public git repository with code	github.com/M1racle1234/my-project
README: project and API description	README.md — API reference
README: security measures in detail	README.md — Security measures
README: SAST/SCA screenshots from Actions	README.md and Appendix D
README: CVE ID and description	README.md — CVE analysis
README: reproduction sequence	Report §7.2 and cve/s2-045/README.md
README: root cause	Report §7.3 and cve/s2-045/README.md
README: fix and proof of elimination	Report §7.4–7.5 and docs/evidence/cve/
Link to last successful pipeline run	Actions run #39226969630 (3/3 green)
Folder with fixes	cve/s2-045/ (Dockerfile.fixed, docker-compose.fixed.yml)
Title page with work, variant, name, HDU number	This document, page 1

9. Conclusion

The first part of the work produced a working, tested API with the three required security properties. SQL injection is prevented structurally by parameterized statements, XSS by output encoding on every serialized field, and broken authentication by bcrypt storage, short-lived signed JWTs and strict middleware. The 59 automated tests and the live smoke test provide repeatable evidence for these claims, and the CI workflow makes the same checks run on every change with Bandit and pip-audit reports attached to each run.

The CVE stage completed the picture. The official OGNL proof produced vulhub: 54289 and a ProcessBuilder payload returned uid=0(root), confirming unauthenticated remote code execution in the container. Upgrading struts2-core from 2.3.30 to 2.3.32 and rebuilding the image removed both effects while the normal upload page still worked. The public repository is live and its first CI run finished green, so every assignment deliverable is now present.

The main lesson is that security controls are only credible when they are exercised by adversarial tests: each protection in this project is paired with a test that tries to break it, and the same discipline was applied to the CVE — the fix was accepted only after the original exploit had been replayed and had failed.

Appendix A. Running the Project

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt

$env:JWT_SECRET = -join ((48..57)+(65..90)+(97..122) |
  Get-Random -Count 64 | ForEach-Object {[char]$_})
$env:SEED_PASSWORD = "ChangeMe#2024"
python run.py

python -m pytest
python scripts/smoke_test.py --base-url http://127.0.0.1:5000
```

Configuration is environment-driven: JWT_SECRET (mandatory, 32+ characters), JWT_TTL_SECONDS, SEED_PASSWORD, DATABASE_PATH, BCRYPT_ROUNDS, HOST and PORT.

Appendix B. Repository Layout

```
app/
  __init__.py      application factory, headers, error handlers
  config.py        environment configuration
  db.py            schema, seed data, connection lifecycle
  security.py      bcrypt, JWT, auth_required decorator
  serializers.py   HTML escaping of all user data
  routes/          auth.py, data.py, notes.py
tests/             59 pytest tests
scripts/           smoke_test.py
cve/s2-045/        Dockerfile.fixed, docker-compose.fixed.yml, CVE record
docs/              CVE plan, evidence, screenshot checklist
.github/workflows/ ci.yml
requirements*.txt  pinned dependencies
run.py             development entry point
```

Appendix C. CVE Experiment Screenshots

The two captures below are the actual console windows from the experiment. They show docker compose ps, the exploit request and the result for the vulnerable build, then the same requests against the fixed build.

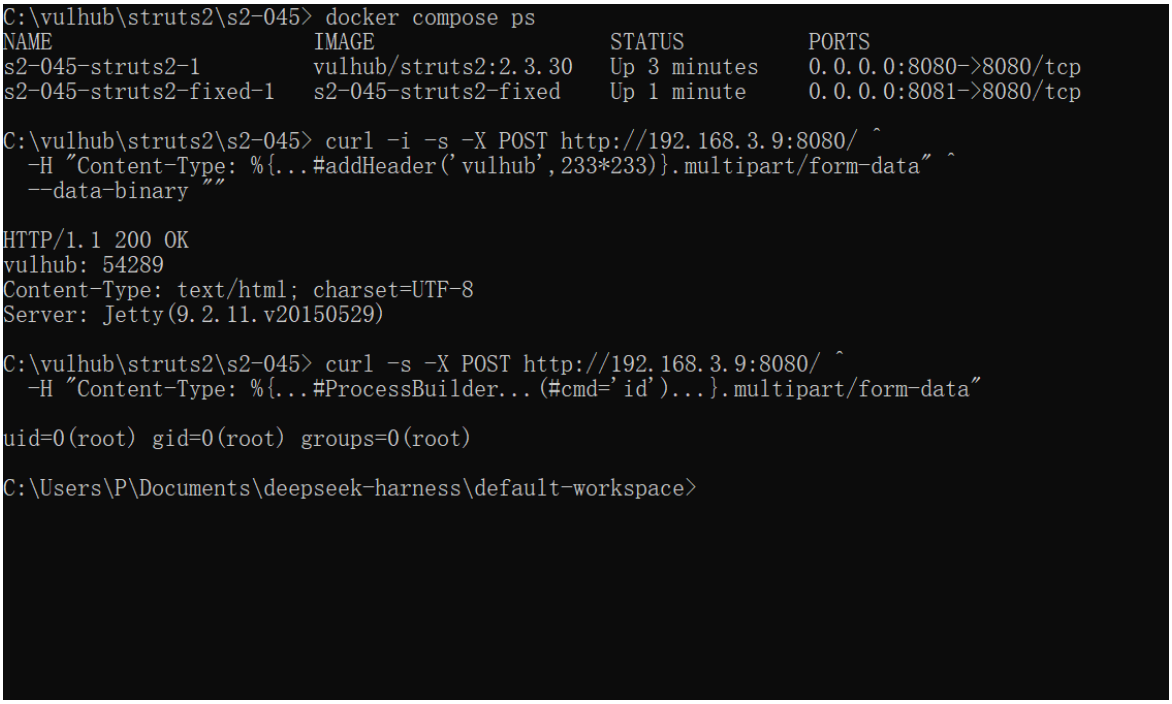


Figure C1. Vulnerable build (2.3.30): vulhub: 54289 and uid=0(root).

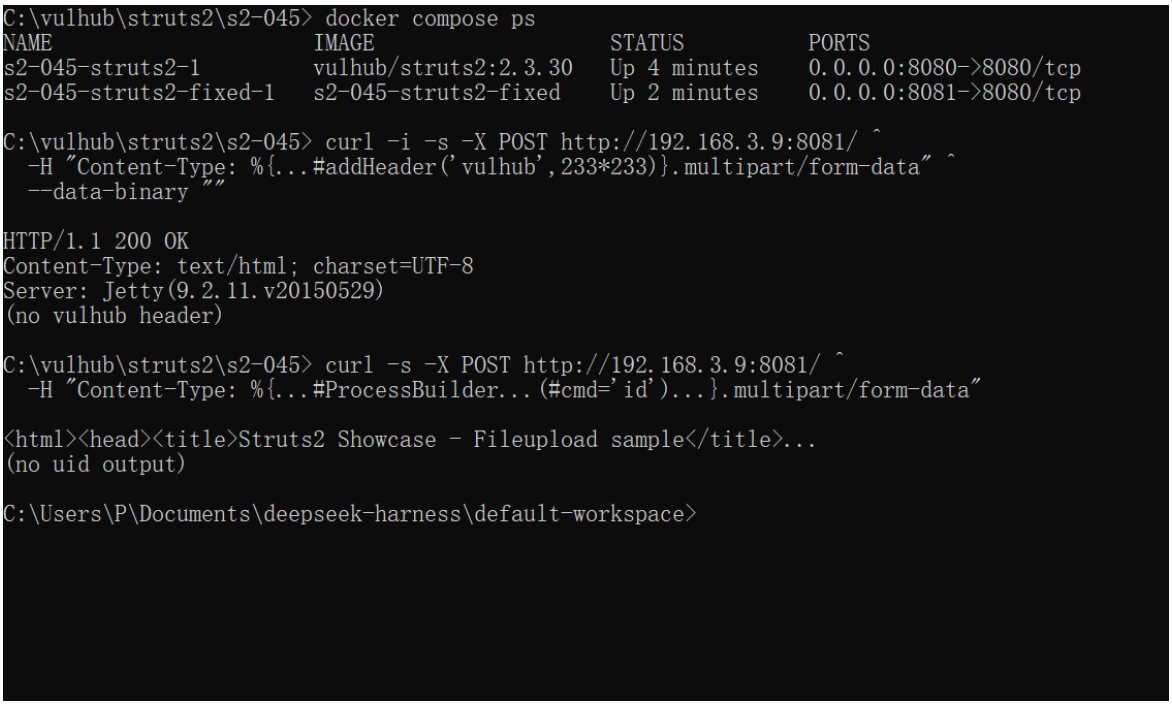


Figure C2. Fixed build (2.3.32): no vulhub header and no command output.

Appendix D. Public CI Run

The repository was pushed on 19 October 2026 and the workflow ran automatically. Run #1 on commit e2fd37b completed with 3/3 jobs green in 42 s. The run page is reproduced below; the complete run is at github.com/M1racle1234/my-project/actions/runs/39226969630.

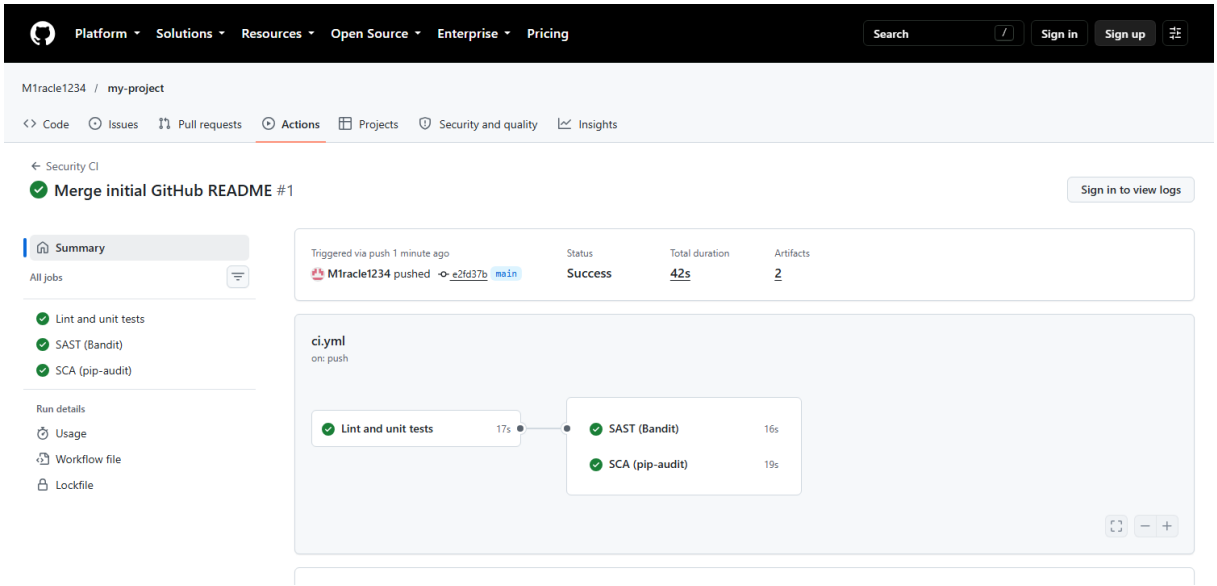


Figure D1. Actions run #1: test, SAST (Bandit) and SCA (pip-audit) all succeeded.

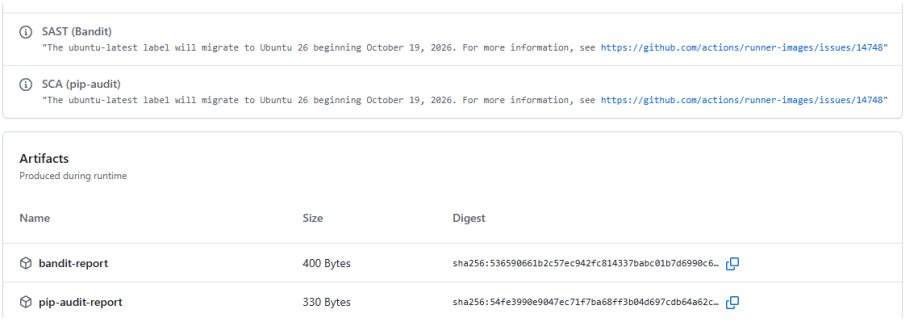


Figure D2. The two report artifacts uploaded by the pipeline.